



University
of Glasgow

A WORLD
TOP 100
UNIVERSITY

Introduction to Data Science and Systems

Lecture 4: **Optimisation I**

- **Parameters, objective functions, classification of optimisation problems**

Dr Nicolas Pugeault

WORLD
CHANGING
GLASGOW

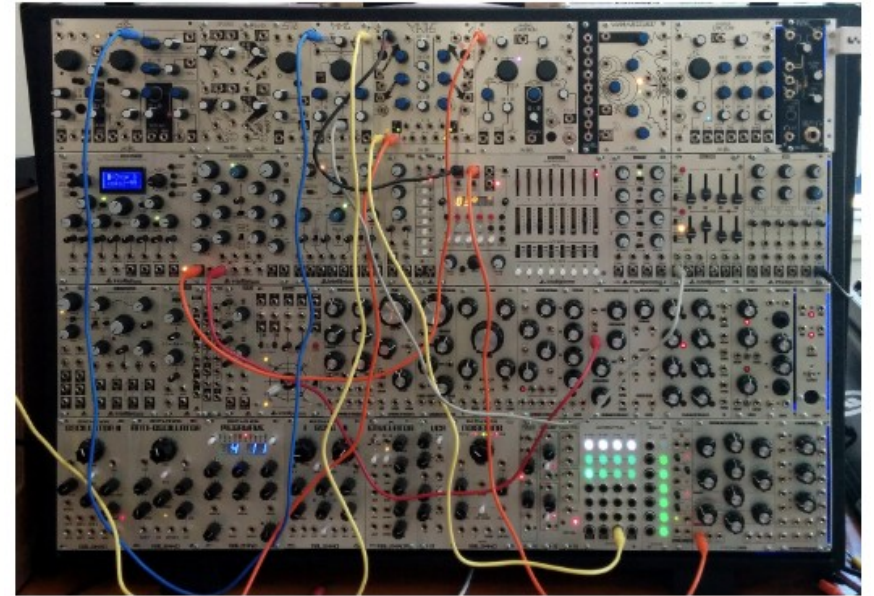
THE SUNDAY TIMES
GOOD
UNIVERSITY
GUIDE
2024
SCOTTISH
UNIVERSITY
OF THE YEAR



What is optimisation?

- **Optimisation** is the process of adjusting things to make them better.
 - we want to do this *automatically* by an algorithm.
 - there are a plethora of algorithms which can then do the automatic adjustment *efficiently*.
 - *optimisation is search*, and optimisation algorithms search efficiently using mathematical structure of the problem space.
 - What is often called "artificial intelligence" often comes down to optimisation.

For electronic music instrument needs



"Enticingly technical synthesizer
1, Control Voltage, Mississippi
Street, Portland, Oregon, USA"

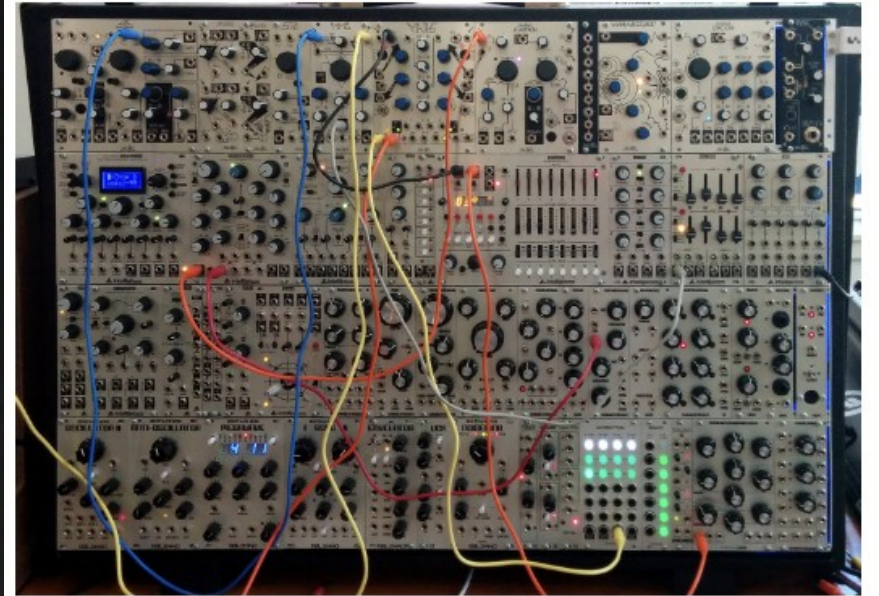


University
of Glasgow

What is optimisation?



For electronic music instrument needs

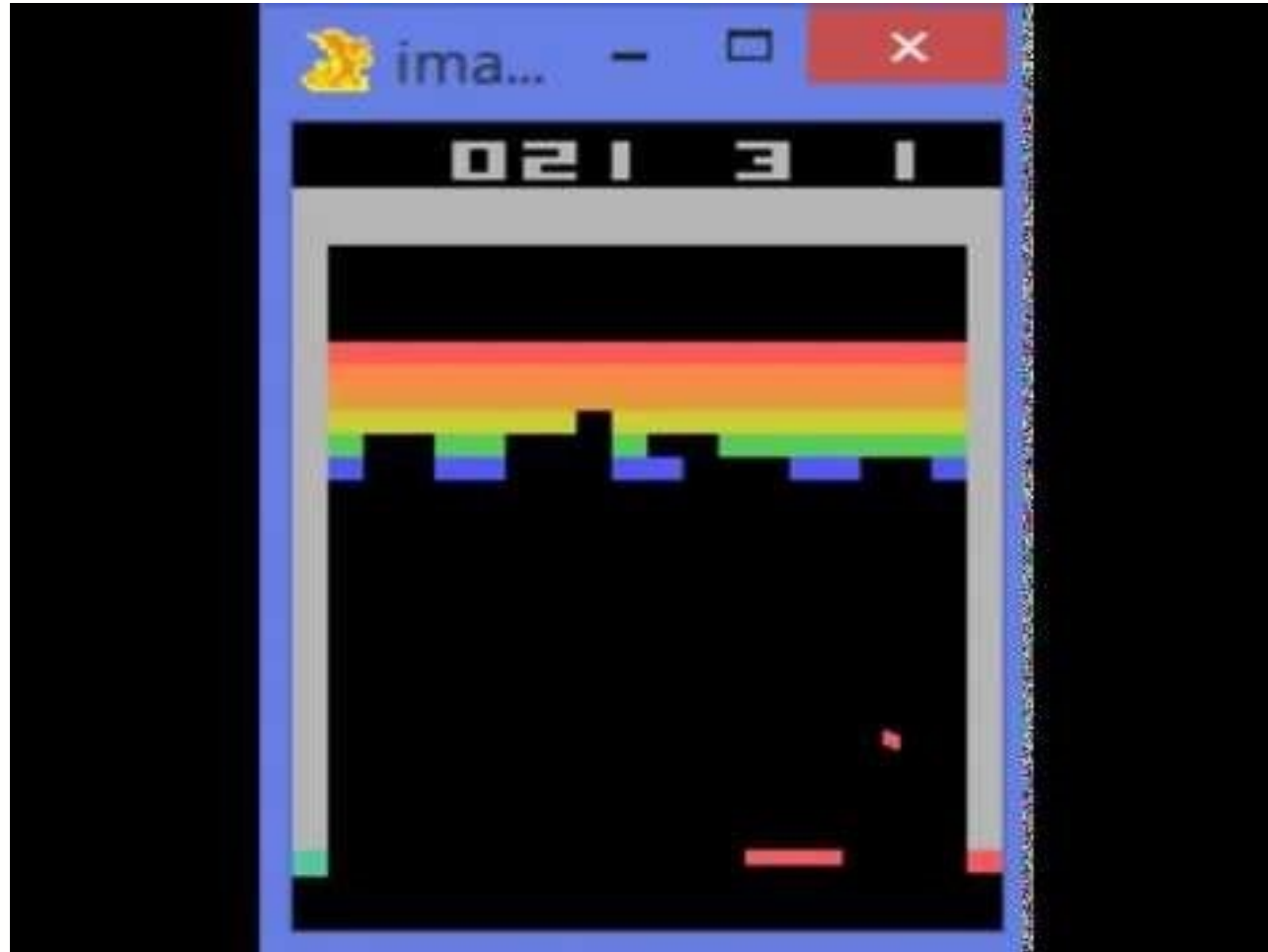


*"Enticingly technical synthesizer
1, Control Voltage, Mississippi
Street, Portland, Oregon, USA"*



University
of Glasgow

Example: Deep Neural Networks



Intended Learning Outcomes

Concepts

- What an objective function, constraint function and a parameter vector is in the context of optimisation
- The difference between discrete and continuous optimisation
- Convex and nonconvex optimisation and how to recognise them
- What constrained optimisation is and the difference between soft and hard constraints
- Key properties of objective functions, including convexity and continuity

Algorithms

- What linear least squares is
- How iterative optimisation works
- The principles of heuristic optimisation
- The properties of random search, with the metaheuristics: locality, memory, temperature and population

Key components of an optimisation problem

- **parameters:** the things we can adjust, which might be a scalar or vector or other array of values, denoted θ .
- **objective function:** a function that maps the parameters onto a *single numerical measure* of how good the configuration is $L(\theta)$.

$$\theta^* = \arg \min_{\theta \in \Theta} L(\theta)$$

θ^* is the configuration that we want to find; the one for which the objective function is lowest.



Discrete vs. continuous

- If the parameters are in a continuous space (typically $\mathbb{R}^n$), the problem is one of **continuous optimization**;
- if the parameters are discrete, the problem is **discrete optimization**.
- Continuous optimisation is usually easier because we can exploit the concept of **smoothness** and **continuity**.





Evaluating the objective function

It may be **expensive** to evaluate the objective function. For example:

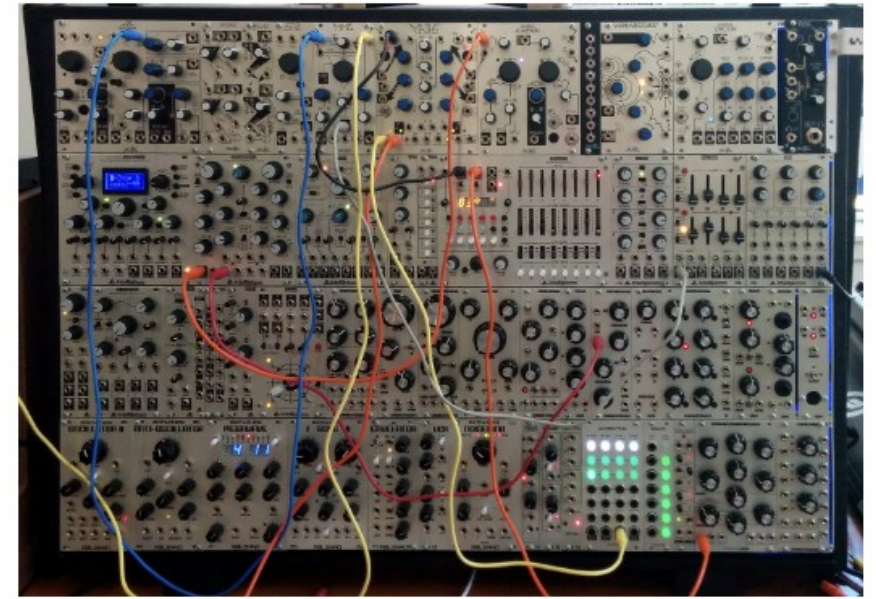
- the computation might take a long time (invert a 10000×10000 matrix);
- or it might require a real-world experiment to be performed (do the users like the new app layout?);
- or it might be dangerous (which wire on the bomb should I cut next?);
- or it might require data that must be bought and paid for (literally expensive).
- mathematical **structure** might help guide the search (analytic solution)





Constraints

- **constraints**: the limitations on the parameters. This defines a region of the parameter space that is feasible, the **feasible set** or **feasible region**.
- For example, the synthesizer has knobs with a fixed physical range, say 0-10; it isn't possible to turn them up to 11.
- Most optimisation problems have constraints of some kind;

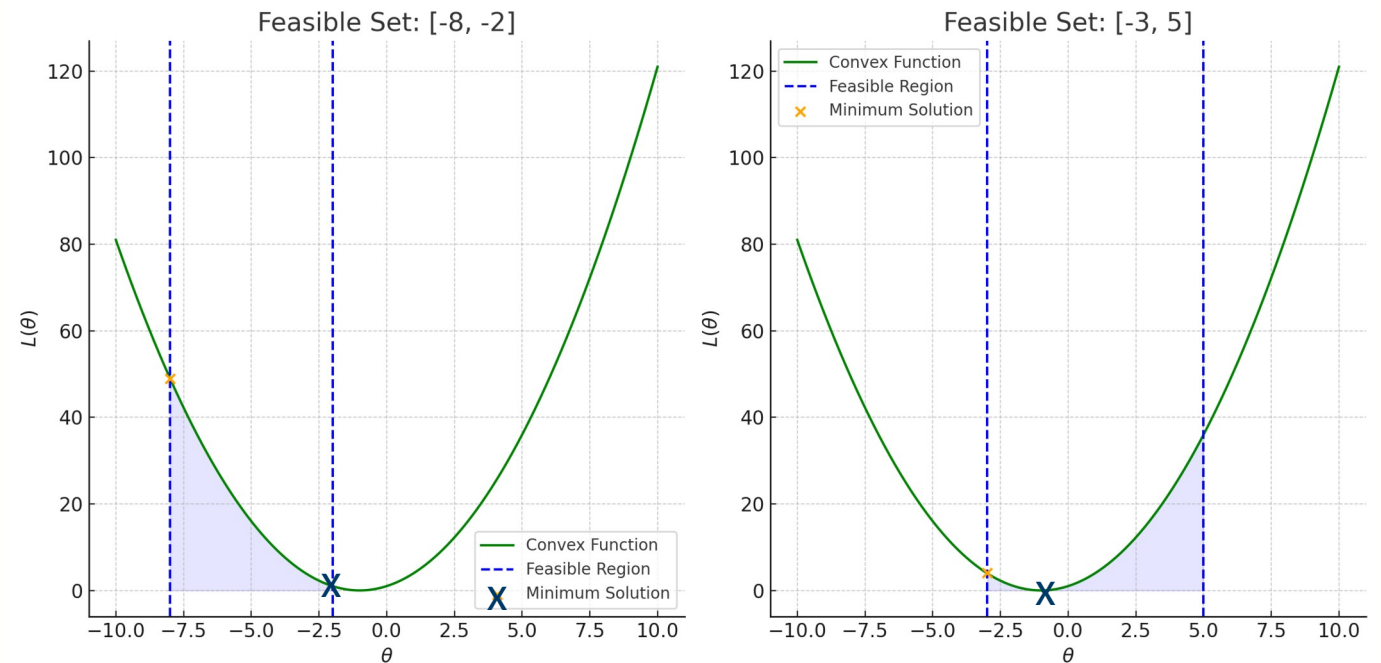




Constraints

- We usually think of the objective function as a **cost** which is *minimised*.
- **Loss function** (more popular)
- We want the cost subject to a certain parameter span, e.g. $[a, b]$

$$\theta^* = \arg \min_{\theta \in \Theta} L(\theta)$$



Focus: continuous optimisation in real vector spaces

This course will focus on optimisation of **continuous** problems in $\mathbb{R}^n$.

$$\theta \in \mathbb{R}^n = [\theta_1, \theta_2, \dots, \theta_n],$$

and the optimisation problem is one of:

$$\theta^* = \arg \min_{\theta \in \mathbb{R}^n} L(\theta), \text{ subject to constraints}$$

Some optimisation algorithms are **iterative**, in that they generate successively better approximations to a solution.

Other methods are **direct**, like linear least squares (which we'll briefly discuss), and involving finding a minimum exactly in one step.

We will focus primarily on **iterative, approximate** optimisation in this course.



Special case: Minimising differences

- It is common to have express problems in a form where the objective function is a **distance between an output and a reference**.
 - Not every objective function has this form, but many do.
 - we have some function $f(x; \theta)$ that produces an output y' from an input x governed by a set of parameters θ ,
 - we measure the difference between the output and some reference y (e.g. using a **vector norm**)

$$\theta^* = \arg \min_{\theta \in \Theta} L(\theta)$$

$$L(\theta) = \|y' - y\| = \|f(\mathbf{x}; \theta) - y\|$$



Special case: Approximation problems

- This is very common in **approximation problems**, where we want to find a function that approximates a set of measured observations.
 - x, y are given (our *data*)
 - We have a function family $f(x; \theta)$ parametrized by θ (our *model*)
 - Find best θ , which minimize $L(\theta)$ (our *loss function*)
- This is the core problem of machine learning.

$$\theta^* = \arg \min_{\theta \in \Theta} L(\theta)$$

$$L(\theta) = \|y' - y\| = \|f(\mathbf{x}; \theta) - y\|$$

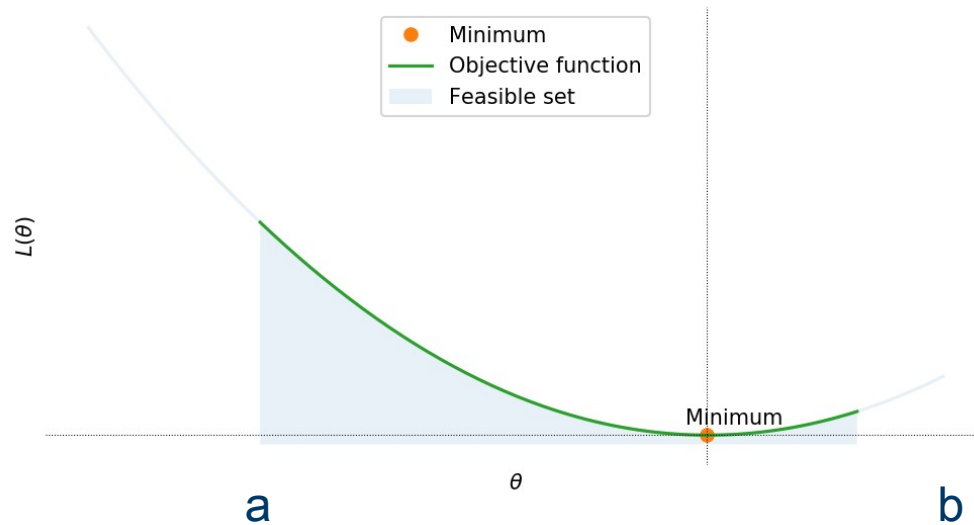


Example 1 - A simple optimisation problem

Example

- L is a square function, e.g. $L(\theta) = \theta^2 + 2\theta + 1$
- Parameter θ
- Subject to: θ in $[a, b]$

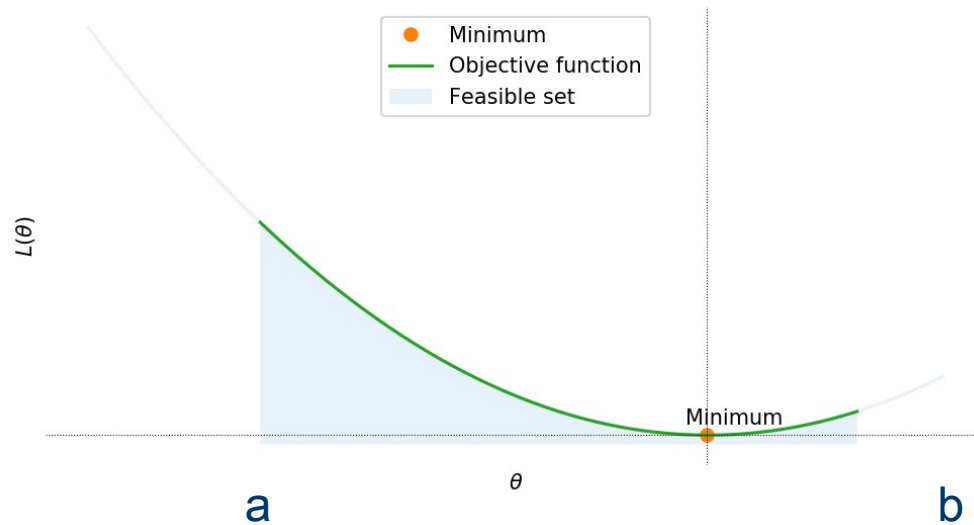
$$\theta^* = \arg \min_{\theta \in \Theta} L(\theta)$$



Example 1 - Analytic solution

Example

- L is a square function, e.g. $L(\theta) = \theta^2 + 2\theta + 1$
- Parameter θ
- Subject to: θ in $[a, b]$



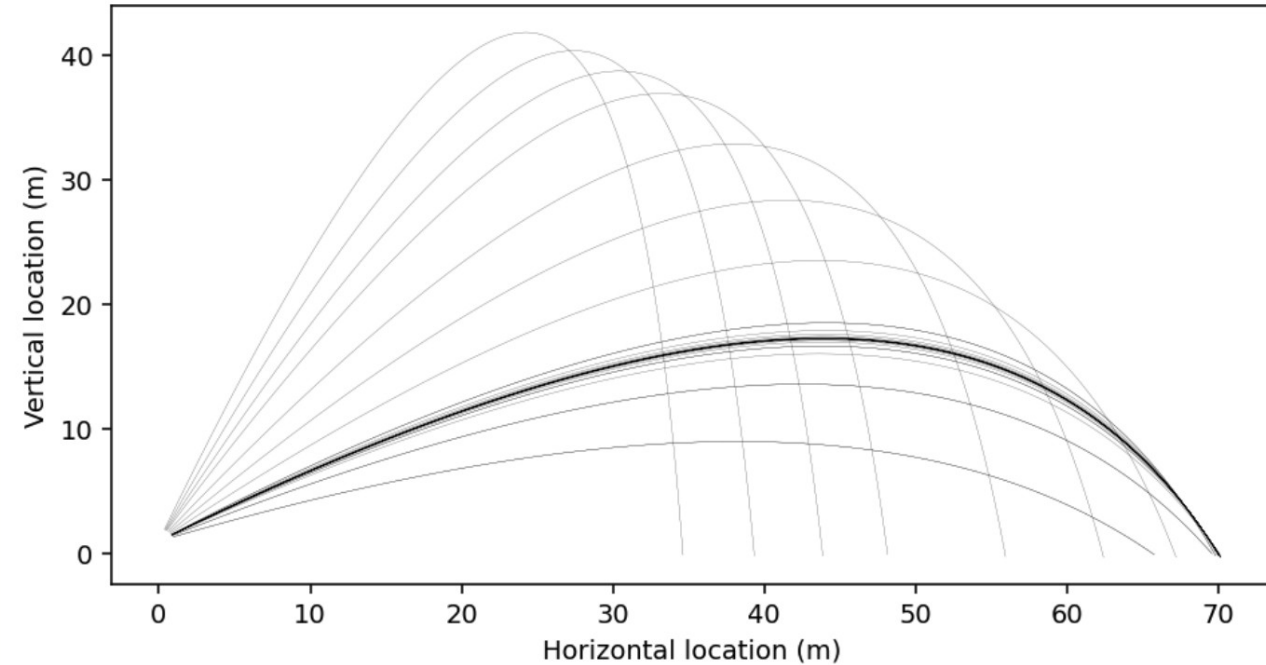
Analytic solution:

- Take the derivative of $L(\theta)$
 - $L'(\theta) = 2\theta + 2$
- Find for what θ it is zero
 - $2\theta + 2 = 0$
 - $\theta = -1$
- To confirm that this is a minimum, check the second derivative:
 - $L''(-1) = 2$
 - Since $L''(-1) > 0$ this indicates a minimum point.
- Hence $\theta^* = -1$



Example 2 - Throwing a stone

- For example, if I wanted to optimise how **far** I could throw a stone, I might be able to adjust the throwing **angle**. This is the *parameter* I could tweak (just one parameter $\theta = [\alpha]$, in this case).
- The objective function must be a function which depends on this parameter.
- I would have to *simulate* throwing the ball to work out how far it went and try and make it go further and further.



Please refer to the Jupyter Notebook for details on how this was plotted.



Example 3 - Geometric median

Problem: Find the median of a $>1D$ dataset.

- We have direct solution:

The standard median is computed by sorting and then selecting the middle element (with various rules for even sized datasets).

- This doesn't work for higher dimensions, and there is no straightforward direct algorithm.

But there is an easy definition of the median: it is the vector that minimises the sum of **distances** to all vectors in the dataset. → optimisation problem

Example 3 - Geometric median

Optimisation problem

- **Parameters:** $\theta = [x, y]$, a position in the plane.
- **Objective function:** the sum of distances between a point and a collection of target points x_i

$$L(\theta) = \sum_i \|\theta - x_i\|_2$$

- The point that minimises this loss is the median!

Optimiser in SciPy: [scipy.optimize](#)

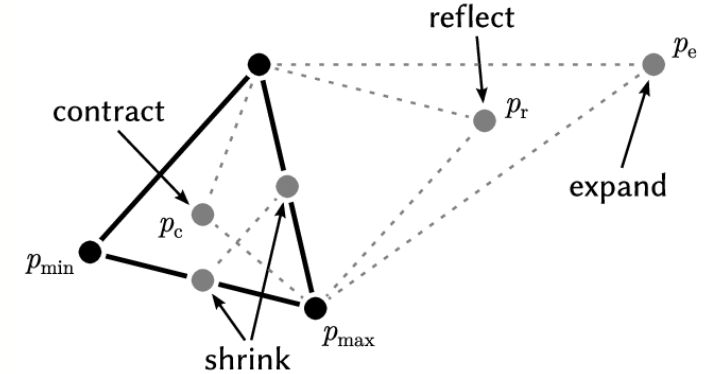
Optimization and root finding (`scipy.optimize`)

SciPy `optimize` provides functions for minimizing (or maximizing) objective functions, possibly subject to constraints. It includes solvers for nonlinear problems (with support for both local and global optimization algorithms), linear programming, constrained and nonlinear least-squares, root finding, and curve fitting.

Nelder-Mead Method

Key Steps:

- 1.Initialisation:** Start with a simplex (triangle in 2D, tetrahedron in 3D, etc.).
- 2.Reflection:** Reflect the worst vertex across the centroid of the others.
- 3.Expansion:** If reflection improves the function, expand further.
- 4.Contraction:** If reflection fails, contract towards the centroid.
- 5.Shrink:** If contraction fails, shrink the simplex towards the best point.



Example 3 - Geometric median

Optimisation problem

- **Parameters:** $\theta = [x, y]$, a position in the plane.
- **Objective function:** the sum of distances between a point and a collection of target points x_i

$$L(\theta) = \sum_i \|\theta - x_i\|_2$$

1. Define loss:

```
def loss(x):  
    # how far are we away from being minimal distance to every point?  
    return np.sum(np.linalg.norm(x[:,None] -  
                                targets.T, axis=1,  
                                ord=np.inf))
```

Example 3 - Geometric median

Optimisation problem

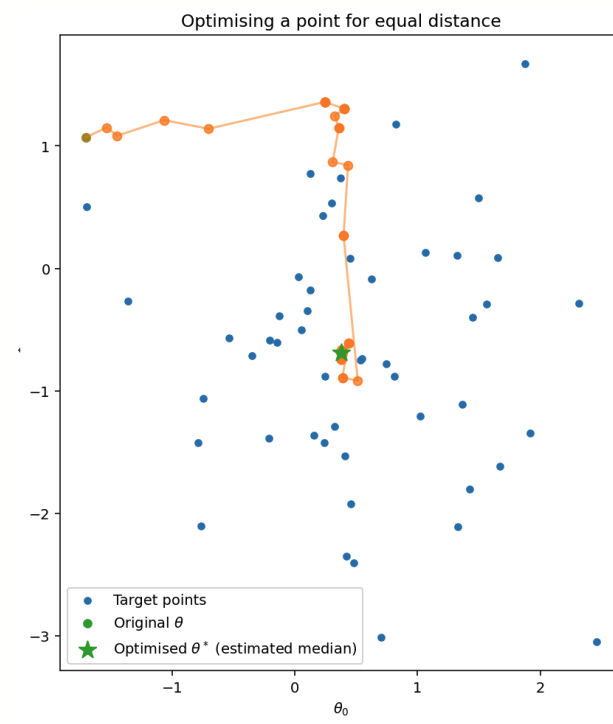
- **Parameters:** $\theta = [x, y]$, a position in the plane.
- **Objective function:** the sum of distances between a point and a collection of target points x_i

$$L(\theta) = \sum_i \|\theta - x_i\|_2$$

1. Minimise the loss using a standard optimizer: :

```
result = scipy.optimize.minimize(loss, theta.ravel(), method='Nelder-Mead',  
                                callback=plot_intermediate, )
```

(theta.ravel()) – returns the flattened 2D points)



Geometric median: an example of optimisation in R2

Optimisation problem:

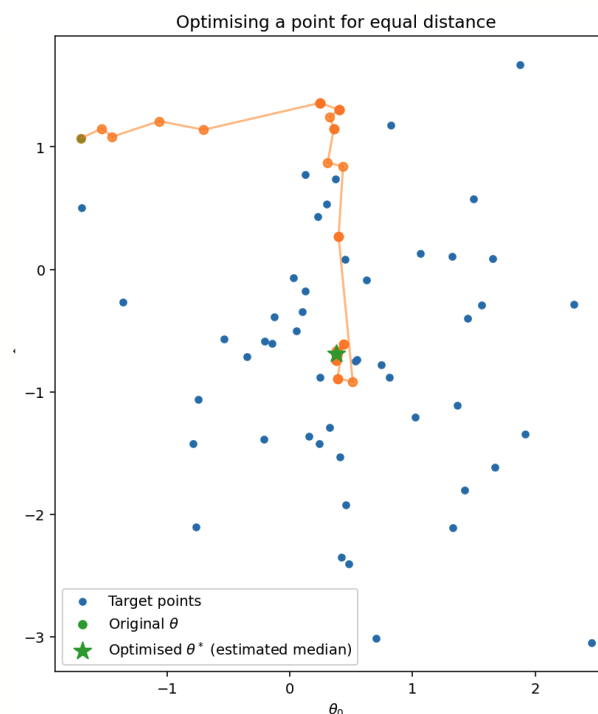
- **parameters** $\theta = [x, y \dots]$, a position in 2D.
- **objective function** the sum of distances between a point and a collection of target points $\mathbf{x}_i$:

$$L(\theta) = \sum_i ||\theta - \mathbf{x}_i||_2$$

2. minimise the loss using a standard optimizer:

```
result = scipy.optimize.minimize(loss, theta.ravel(), method='Nelder-Mead',  
                                callback=plot_intermediate, )
```

`theta.ravel()` – return the flatten random 2D points





Properties of optimisation

Every optimisation problem has two parts:

- **Parameters**, the things that can be adjusted.
- **Objective function**, which measures how good a particular set of parameters are.

An optimisation problem usually also has:

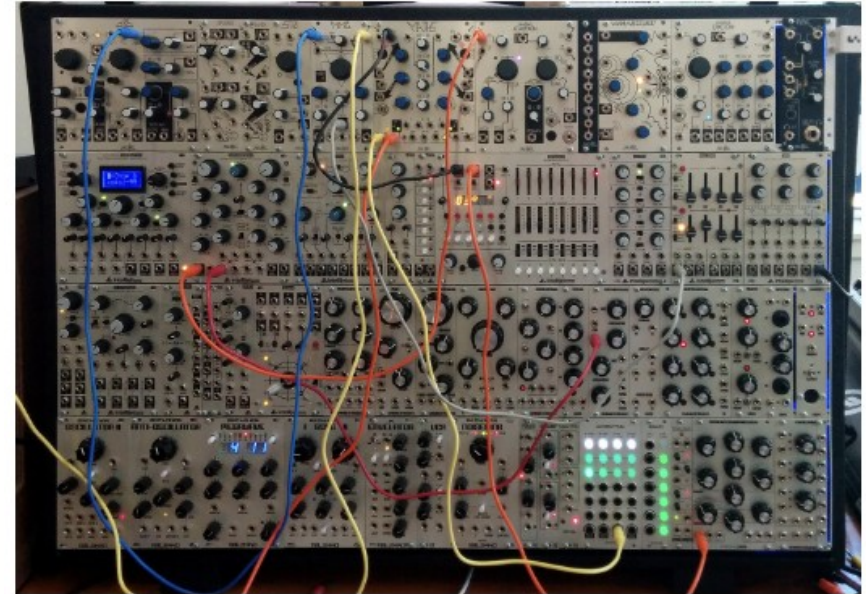
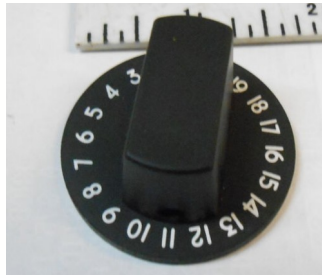
- **Constraints**, that define the feasible set of parameters.

The **objective function** is a function *of the parameters* which returns a *single scalar value*, representing how good that parameter set is.



Constrained optimisation

- If a problem has constraints on the parameters beyond purely minimising the objective function then the problem is **constrained optimisation**.
- For example, in the synthesizer above, it's not much use if the optimal solution requires one the knobs to be turned to an impossible value; the parameter space is limited in extent on every dimension.
- This limits the **feasible set** of the parameters.





Constrained optimisation

A constrained optimisation might be written in terms of an equality constraint:

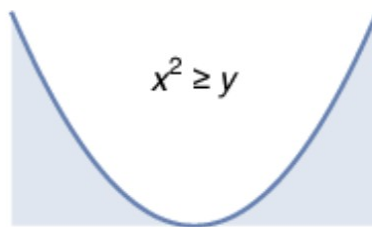
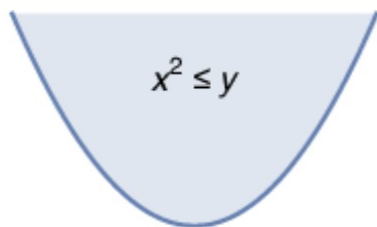
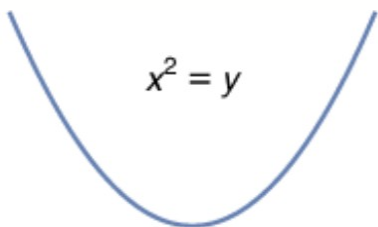
$$\theta^* = \arg \min_{\theta \in \Theta} L(\theta) \text{ subject to } c(\theta) = 0,$$

or an inequality:

$$\theta^* = \arg \min_{\theta \in \Theta} L(\theta) \text{ subject to } c(\theta) \leq 0,$$

where $c(\theta)$ is a function that represents the constraints.

- An **equality** constraint can be thought of as constraining the parameters to a *surface*, to represent a tradeoff.
 - For example, $c(\theta) = \|\theta\|_2 - 1$ forces the parameters to lie on the surface of a unit sphere.
- An **inequality** constraint can be thought of as constraining the parameters to a *volume*, to represent bounds on the values.
 - For example, $c(\theta) = \|\theta\|_\infty - 10$ forces the parameters to lie within a box extending $(-10, 10)$ around the origin -- perhaps the range of the knobs on the synthesizer.





Constrained optimisation

A constrained optimisation might be written in terms of an equality constraint:

$$\theta^* = \arg \min_{\theta \in \Theta} L(\theta) \text{ subject to } c(\theta) = 0,$$

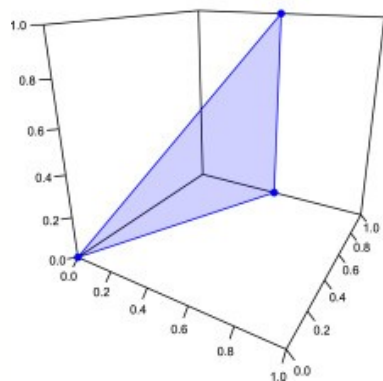
or an inequality:

$$\theta^* = \arg \min_{\theta \in \Theta} L(\theta) \text{ subject to } c(\theta) \leq 0,$$

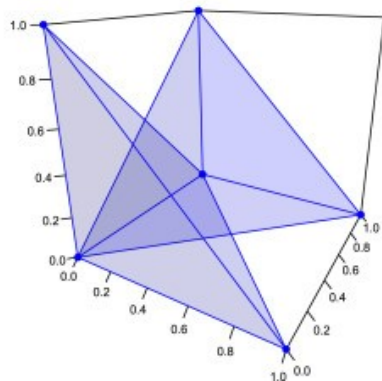
where $c(\theta)$ is a function that represents the constraints.

- An **equality** constraint can be thought of as constraining the parameters to a *surface*, to represent a tradeoff.
 - For example, $c(\theta) = \|\theta\|_2 - 1$ forces the parameters to lie on the surface of a unit sphere.
- An **inequality** constraint can be thought of as constraining the parameters to a *volume*, to represent bounds on the values.
 - For example, $c(\theta) = \|\theta\|_\infty - 10$ forces the parameters to lie within a box extending $(-10, 10)$ around the origin -- perhaps the range of the knobs on the synthesizer.

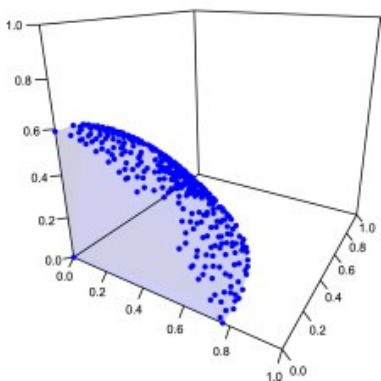
(A) Equality and Inequality Constraints



(B) Non-Convex Inequality Constraints



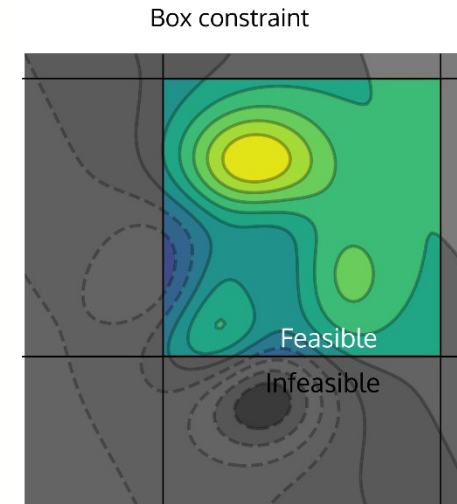
(C) Non-Linear Inequality Constraints





Common constraint types

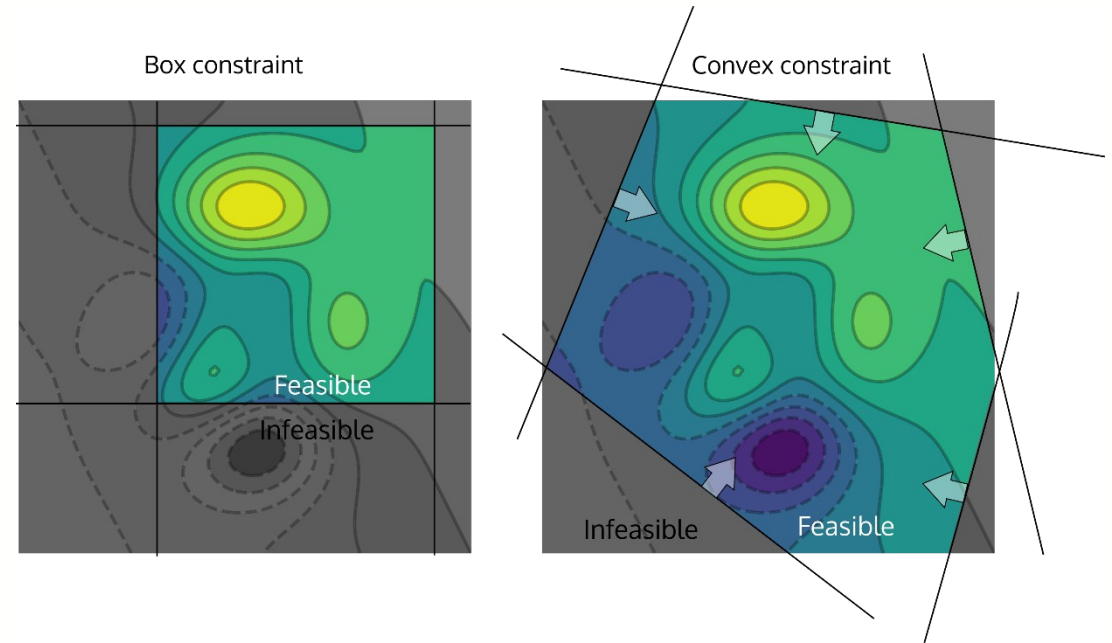
- A **box constraint** is a simple kind of constraint, and is just a requirement that θ lie within a box inside $\mathbb{R}^2$ (or $\mathbb{R}^n$);
 - for example, that every element $0 < \theta_i < 1$ (all parameters in the positive unit cube)
 - or $\theta_i > 0$ (all parameters in the positive **orthant**).
- This is an **inequality** constraint with a simple form of $c(\theta)$.
- Many optimisation algorithms support box constraints.





Common constraint types

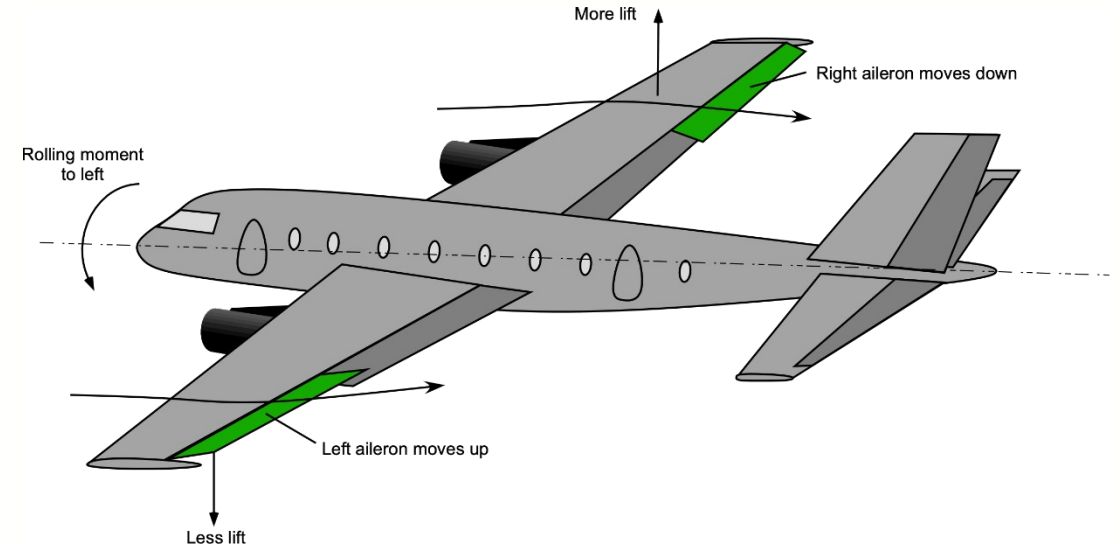
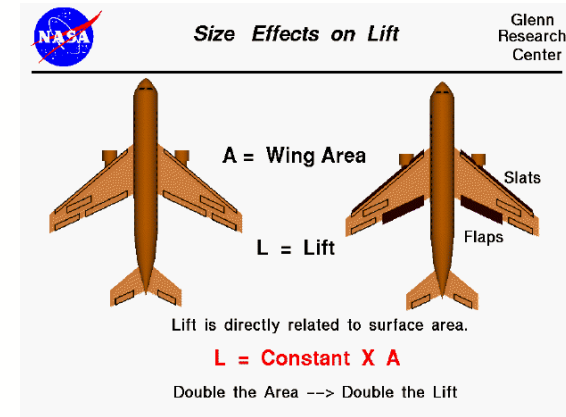
- A **convex constraint** is another simple kind of constraint, where the constraint is a collection of inequalities on a convex sum of the parameters θ .
- **Box constraints are a specific subclass of convex constraints.**





Common constraint types

- **Unconstrained optimization** does not apply any constraints to the parameters, and any parameter configuration in the search space is possible.
- In many problems, pure unconstrained optimisation will lead to **unhelpful** results (-- which might be true but impossible to construct).



"the airplane will get the best lift if the wing is two hundred miles long"



Constraints and penalties

- Unconstrained optimisation rarely gives useful answers on its own. Consider the example of the airfoil. Increasing lift might be achieved by making the airfoil length longer and longer. At some point, this might become physically impossible to build.
- Although we often represent θ as being in $\mathbb{R}^n$, the feasible set is typically not the entire vector space.
- For optimisation problem with constraints, there are two approaches to deal with this:
 - **Constrained optimisation**
 - **Soft constraints**



Hard constraints

Use an optimisation algorithm that **supports hard constraints inherently**.

- This is straightforward for certain kinds of optimisation, but trickier for general optimisation.
- Typically, constraints will be specified as either a **convex region** or a simple (hyper)rectangular region of the space (a **box constraint**).

• Pros:

- Guarantees that solution will satisfy constraints.
- May be able to use constraints to speed up optimisation.

• Cons:

- may be less efficient than unconstrained optimization (**convexity**).
- Fewer algorithms available for optimisation.
- may be hard to specify feasible region with the parameters available in the optimiser.



Soft constraints

Apply penalties to the **objective function** to "**discourage**" solutions that violate the constraints:

$$L(\theta') = L(\theta) + \lambda(\theta),$$

where $\lambda(\theta)$ is a **penalty function** with an increasing value as the constraints are more egregiously violated.

- This is particularly appropriate if the constraints really are soft (it doesn't perhaps matter if the maximum airfoil length is 1.5m or 1.6m, but it can't be 10m).

- **In this case, the penalties are just terms added to the**

- | | |
|---|--|
| <ul style="list-style-type: none">• penalty function is modified. The optimiser stays the same, but the objective function is modified<ul style="list-style-type: none">• can deal with <i>soft</i> constraints sensibly | <ul style="list-style-type: none">• may not respect important constraints, particularly if they are very sharp• can be hard to formulate constraints as penalties• cannot take advantage of efficient search in constrained regions of space |
|---|--|

Relaxation of objective functions

Penalisation

- Terms which augment an **objective function** to minimise some other property of the solution, typically to approximate constrained optimisation.

Relaxation

- approximation of a difficult problem by a nearby problem that is easier to solve.
- E.g., A linear programming relaxation of an integer programming problem removes the integrality constraint and so allows non-integer rational solutions

Lagrange multipliers:

A Lagrangian relaxation of a complicated problem in combinatorial optimization penalizes violations of some constraints, allowing an easier relaxed problem to be solved.

$$\mathcal{L}(x, \lambda) = f(x) - \lambda g(x)$$

$\mathcal{L}$ = Lagrangian

λ = Lagrange multiplier

$g(x)$ = equality constraint

$f(x)$ = function

x = integer



Properties of optimisation

Every optimisation problem has two parts:

- **Parameters**, the things that can be adjusted.
- **Objective function**, which measures how good a particular set of parameters are.

An optimisation problem usually also has:

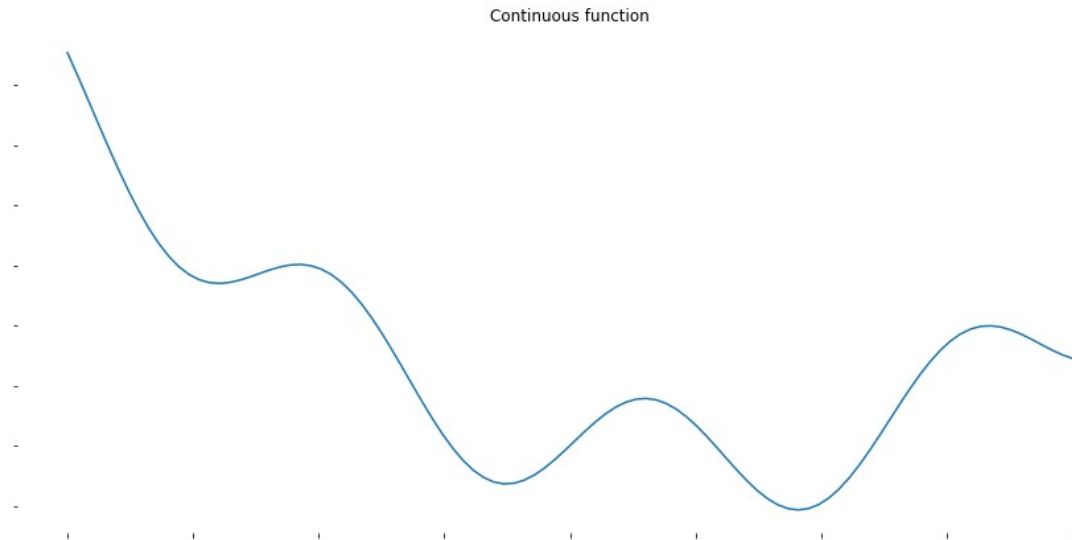
- **Constraints**, that define the feasible set of parameters.

The **objective function** is a function *of the parameters* which returns a *single scalar value*, representing how good that parameter set is.



Properties of the objective function: **Continuity**

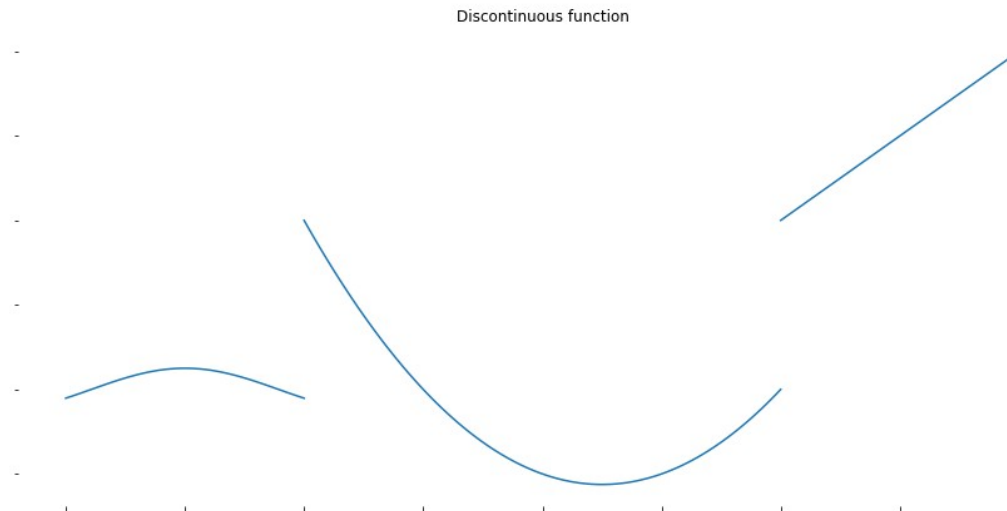
- An **objective** function is **continuous** if for some very small adjustment to θ there is an *arbitrarily* small change in $L(\theta)$. This means that there will never be "nasty surprises" if we move slowly enough through the space of θ ; no sudden jumps in value.





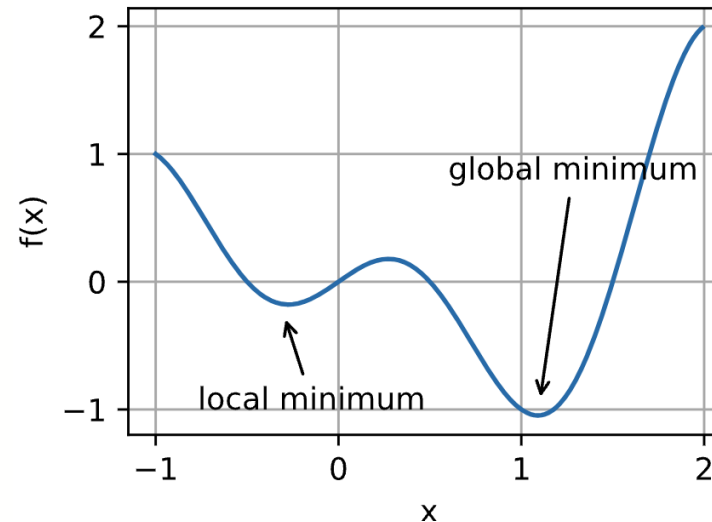
Continuity

- If a function is discontinuous, local search methods are **not guaranteed** to converge to a solution.
- Optimisation for discontinuous objective functions is typically much **harder** than for continuous functions. This is because there could be arbitrary changes in the objective function for any adjustment to the parameters.



Properties of the objective function: Local and global minima

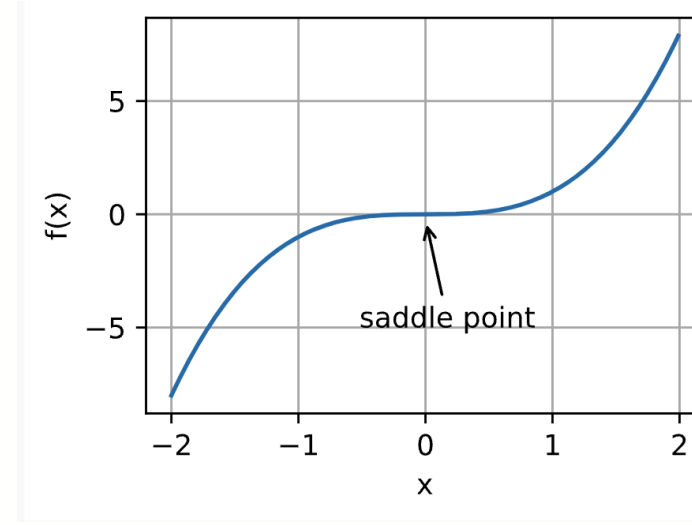
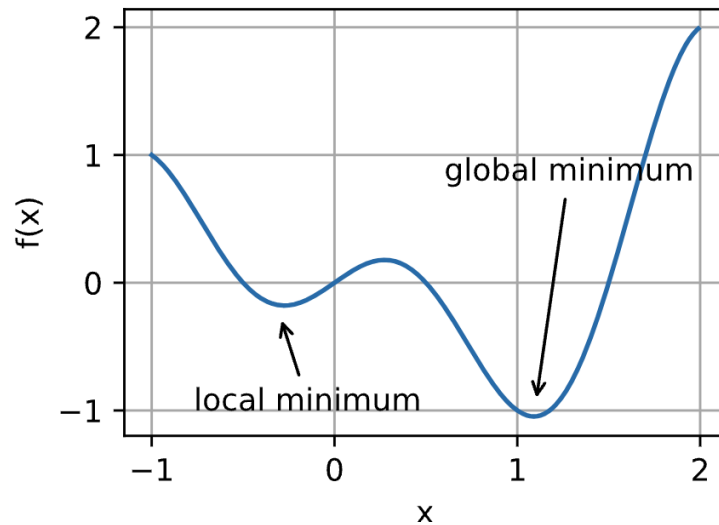
- **Local Minima** is any point where the objective functions increases in every direction around that point (that parameter setting).
- **Global Minima** is the smallest overall value of a set, function, etc., over its entire range.



- **Iteration optimisation** may only minimize the objective function *locally*, rather than *globally*, as the gradient of the objective function's solutions approaches or becomes zero (vanish). (will be covered later)

Properties of the objective function: Saddle points

- Besides *local minima*, **saddle points** are another reason for gradients to vanish.
- A *saddle point* is any location where all gradients of a function vanish but which is neither a global nor a local minimum.

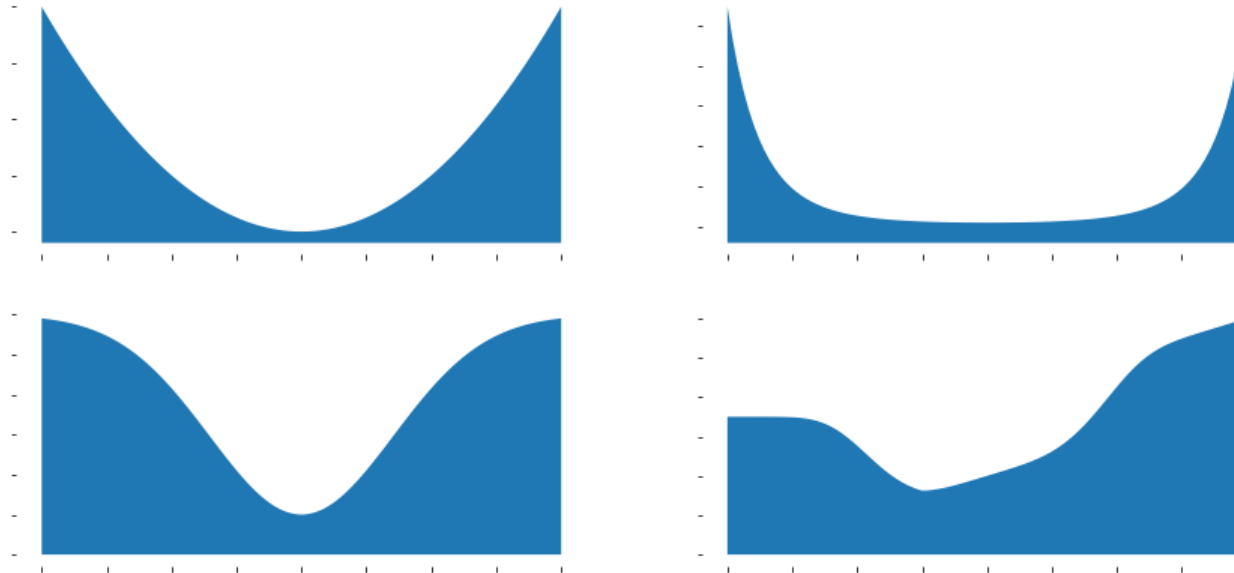




Properties of the objective function: Convexity

- **Convexity** implies that finding any **minimum** is equivalent to finding the **global** minimum -- the guaranteed best possible solution.
- In a convex problem, if we find a minimum, we can stop searching. If we can show there *is no minimum*, we can also stop searching.

Some convex functions

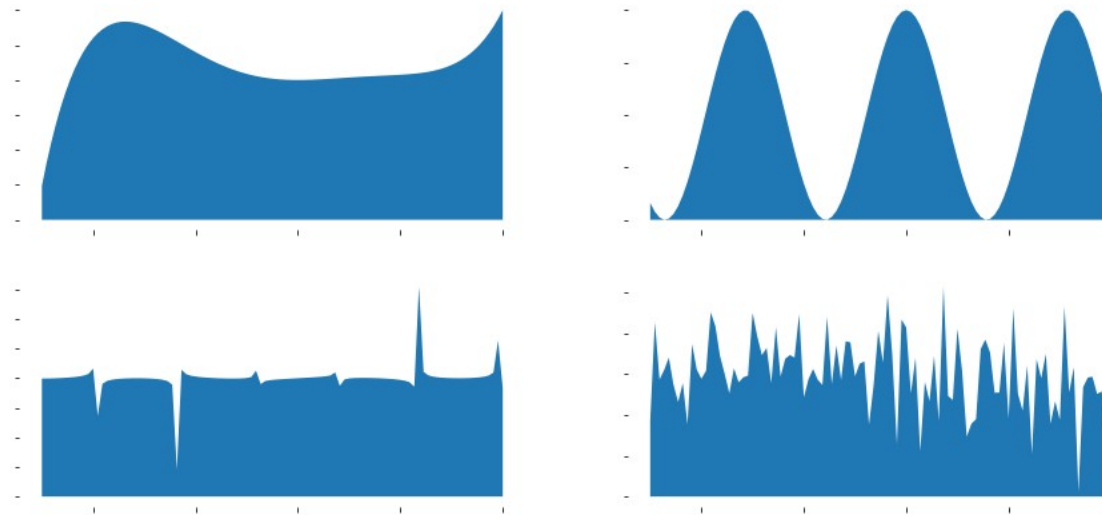




Properties of the objective function: Convexity

- **Convexity** implies that finding any **minimum** is equivalent to finding the **global** minimum -- the guaranteed best possible solution.
- In a convex problem, if we find a minimum, we can stop searching. If we can show there *is no minimum*, we can also stop searching.

Non-convex functions





Convexity

Convex Sets

- Simply put, a set X in a vector space is *convex* if for **ANY** $a, b \in X$ the **line segment** connecting a and b is also in X . In mathematical terms this means that for all $\lambda \in [0, 1]$ we have

$$\lambda a + (1 - \lambda)b \in \mathcal{X} \text{ whenever } a, b \in \mathcal{X}.$$

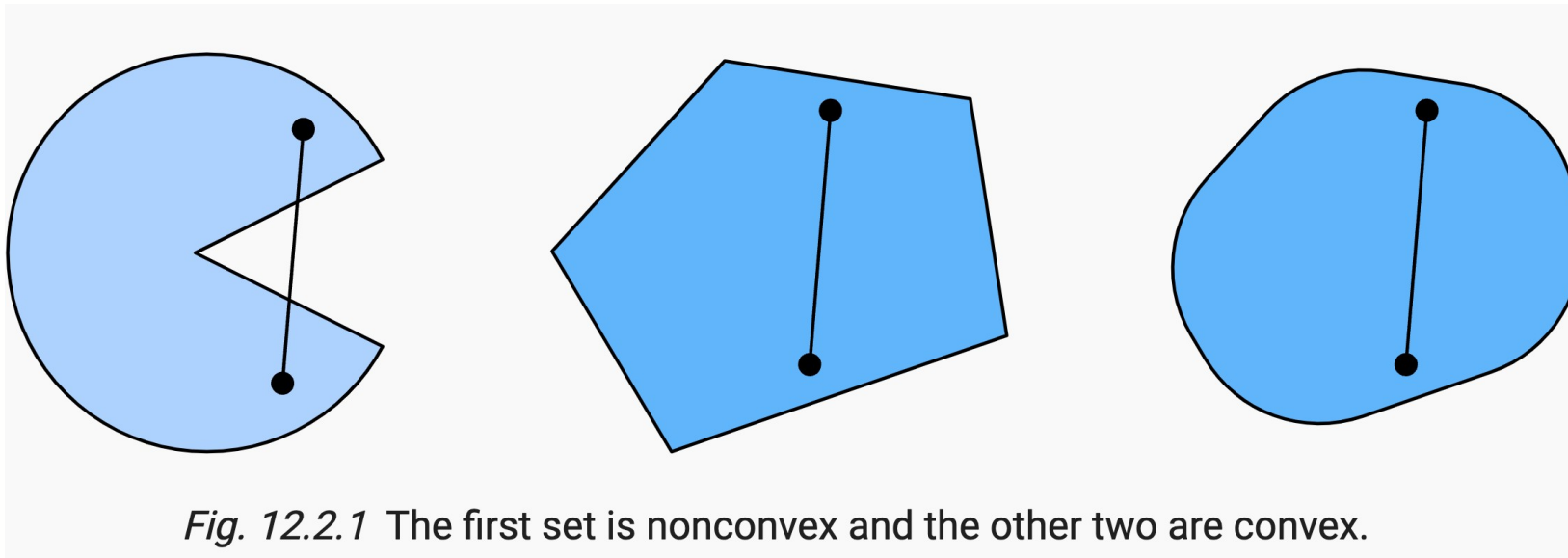
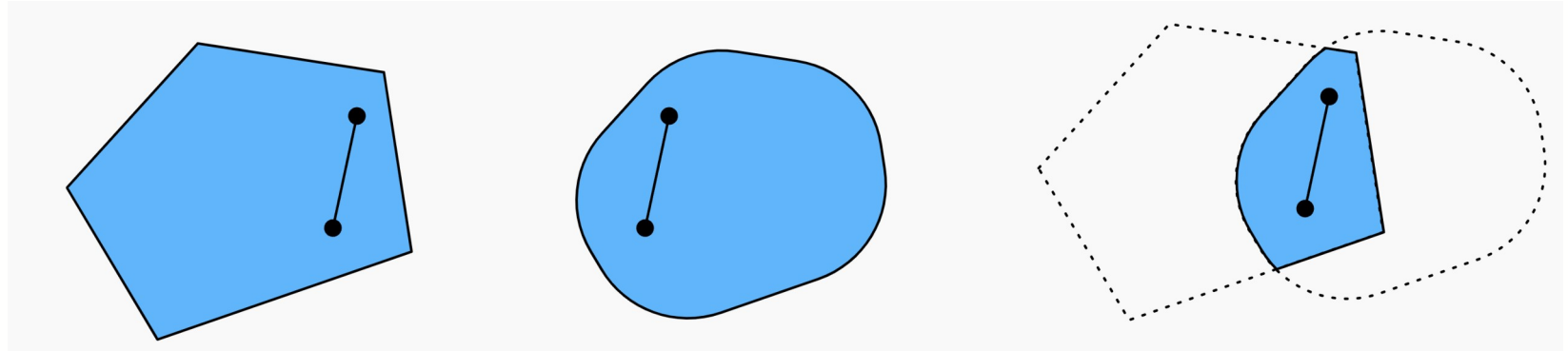


Fig. 12.2.1 The first set is nonconvex and the other two are convex.

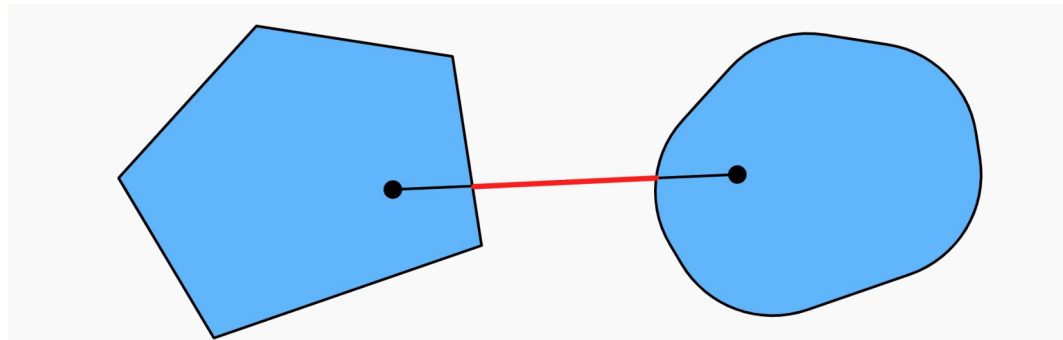


Convexity

- The intersection between two convex sets is convex.



- The union of two convex sets need not be convex.

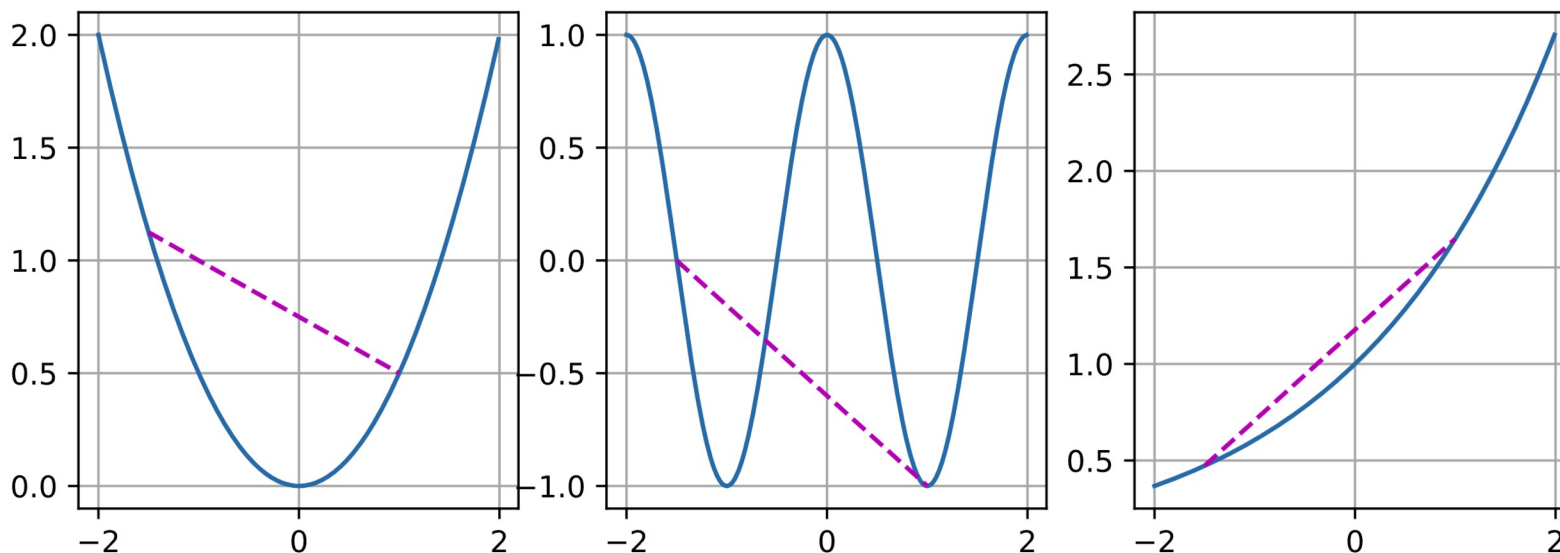




Convex Functions

Now that we have convex sets we can introduce *convex functions* f . Given a convex set X , a function $f: X \rightarrow \mathbb{R}$ is *convex* if for **all** $x, x' \in X$ and for all $\lambda \in [0, 1]$ we have

$$\lambda f(x) + (1 - \lambda)f(x') \geq f(\lambda x + (1 - \lambda)x').$$



Properties of the Convex functions

P1. Local Minima Are Global Minima

We can prove it by contradiction, see: https://d2l.ai/chapter_optimization/convexity.html.

It means that if we minimize functions we cannot “get stuck”.

Note, though, that this does not mean that there cannot be more than one global minimum or that there might even exist one. (A line)



If the objective function is **convex** *and* any constraints form **convex** portions of the search space, then the problem is **convex optimisation**.

Methods:

1. Linear Programming (**scipy.optimize.linprog**):

1. Objective function and constraints are linear.
2. Example: Maximising profit given resource constraints.

2. Quadratic Programming (**supported by scipy.optimize.minimize**):

1. Quadratic objective function with linear constraints.
2. Example: Portfolio optimisation where risk is minimised given expected returns.

3. Specialised Cases:

1. **Semi-Quadratic Programming**: Involves semi-definite constraints.
2. **Quadratically Constrained Quadratic Program**: Quadratic objective and quadratic constraints.
3. Example: Optimal control problems in engineering.

Nonconvex problems require the use of **iterative optimisation** methods



Example: Linear Least Squares

- An example is **linear least squares**, which solves objective functions of the form:

$$\arg \min_x L(\mathbf{x}) = \|A\mathbf{x} - \mathbf{y}\|_2^2,$$

- It finds x that is closest to the solution $Ax=y$ in the sense of minimising the squared L2 norm. The squaring of the norm just makes the algebra easier to derive.
- This equation is **convex** -- it is a **quadratic** function and even in multiple dimensions it must have a **single, global minimum**, which can be found directly.
- Quadratic functions only ever have zero or one minimum.



Direct convex optimisation: least squares

- **Direct Solution : normal equations methods**
Optimal: $A\mathbf{x}=\mathbf{y}$

$$\arg \min_x L(\mathbf{x}) = \|A\mathbf{x} - \mathbf{y}\|_2^2,$$

$$(A^\top A) \mathbf{x} = A^\top \mathbf{y}$$

and therefore our solution is

$$\mathbf{x}^* = (A^\top A)^{-1} A^\top \mathbf{y}$$

which can also be written as

$$\mathbf{x}^* = A^+ \mathbf{y}$$

where A^+ is the **Pseudo-Inverse** of A .

Example - normal equations methods

- Given

$$A = \begin{pmatrix} \frac{1}{2} & 0 \\ 0 & \frac{1}{2} \end{pmatrix}, \quad b = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$

- Use **normal equations methods** to solve the below optimization task:

$$\min_x \|Ax - b\|_2^2$$

Hint

$$\arg \min_x L(\mathbf{x}) = \|A\mathbf{x} - \mathbf{y}\|_2^2,$$

$$(A^\top A) \mathbf{x} = A^\top \mathbf{y}$$

and therefore our solution is

$$\mathbf{x}^* = (A^\top A)^{-1} A^\top \mathbf{y}$$

which can also be written as

$$\mathbf{x}^* = A^+ \mathbf{y}$$

where A^+ is the **Pseudo-Inverse** of A .

Example - normal equations methods

Solution

We want to minimize the least squares objective:

$$\min_x \|Ax - b\|_2^2 \quad (2)$$

where:

$$A = \begin{pmatrix} \frac{1}{2} & 0 \\ 0 & \frac{1}{2} \end{pmatrix}, \quad b = \begin{pmatrix} 1 \\ 1 \end{pmatrix} \quad (3)$$

Step 1: Form the Normal Equations

The normal equations are given by:

$$A^\top Ax = A^\top b \quad (4)$$

Step 2: Compute $A^\top A$ and $A^\top b$

First, calculate $A^\top A$:

$$A^\top = A = \begin{pmatrix} \frac{1}{2} & 0 \\ 0 & \frac{1}{2} \end{pmatrix} \quad (5)$$

$$A^\top A = \begin{pmatrix} \frac{1}{2} & 0 \\ 0 & \frac{1}{2} \end{pmatrix} \begin{pmatrix} \frac{1}{2} & 0 \\ 0 & \frac{1}{2} \end{pmatrix} = \begin{pmatrix} \frac{1}{4} & 0 \\ 0 & \frac{1}{4} \end{pmatrix} \quad (6)$$

Next, calculate $A^\top b$:

$$A^\top b = \begin{pmatrix} \frac{1}{2} & 0 \\ 0 & \frac{1}{2} \end{pmatrix} \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} \frac{1}{2} \\ \frac{1}{2} \end{pmatrix} \quad (7)$$

Step 3: Solve the Normal Equations using the Inverse

Now, the normal equations are:

$$\begin{pmatrix} \frac{1}{4} & 0 \\ 0 & \frac{1}{4} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} \frac{1}{2} \\ \frac{1}{2} \end{pmatrix} \quad (8)$$

Since $A^\top A = \begin{pmatrix} \frac{1}{4} & 0 \\ 0 & \frac{1}{4} \end{pmatrix}$ is a diagonal matrix, we can easily compute its inverse:

$$(A^\top A)^{-1} = \begin{pmatrix} 4 & 0 \\ 0 & 4 \end{pmatrix} \quad (9)$$

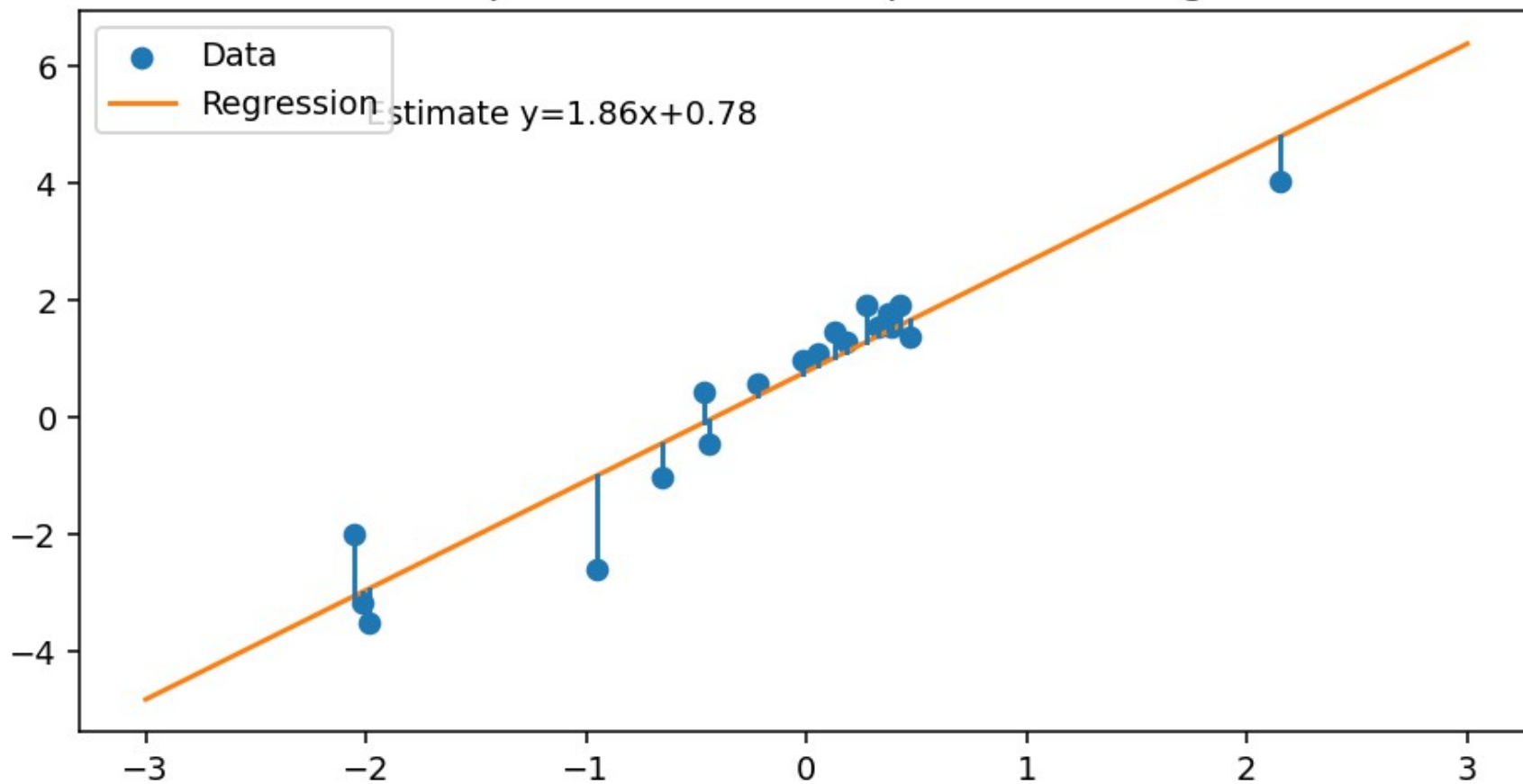
Now, multiply both sides of the normal equation by $(A^\top A)^{-1}$:

$$x = (A^\top A)^{-1} A^\top b = \begin{pmatrix} 4 & 0 \\ 0 & 4 \end{pmatrix} \begin{pmatrix} \frac{1}{2} \\ \frac{1}{2} \end{pmatrix} = \begin{pmatrix} 4 \times \frac{1}{2} \\ 4 \times \frac{1}{2} \end{pmatrix} = \begin{pmatrix} 2 \\ 2 \end{pmatrix}$$
$$x^* = \begin{pmatrix} 2 \\ 2 \end{pmatrix}$$



Application: Line fitting

Some data points and a least-squares linear regression





Application: Line fitting

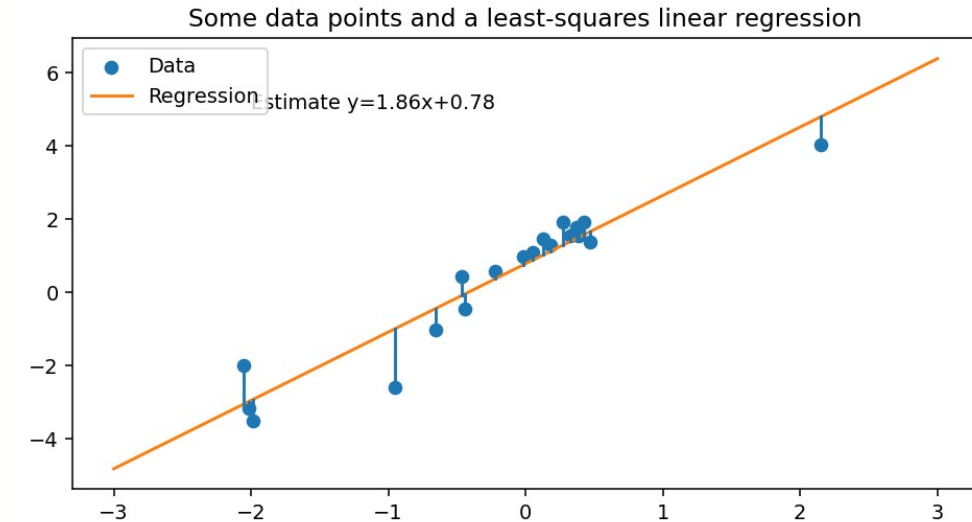
We will examine this process for the simplest possible **linear regression** example: finding gradient m and offset c for the line equation :

$$y = mx + c$$

such that the squared distance to a set of observed (x,y) data points is minimised.

$$L(\theta) = \sum_i (y - mx_i - c)^2$$

This is a search over the $\theta=[m,c]$ space, and the objective function is:



This is a search over the $\theta=[m,c]$ space, and the objective function is:

$$L(\theta) = \sum_i (y - mx_i - c)^2$$

- We can solve this directly using the **pseudo-inverse** via the SVD. This is a problem that can be solved directly in one step.

```
## Solution via SVD
# Using the "Vandermonde matrix" approach
## NOTE: You don't have to understand how this code works

A = np.stack([line_x ** 0, line_x ** 1]).T
A_plus = np.linalg.pinv(A)
regression = A_plus @ line_y
```

$$(A^T A) \mathbf{x} = A^T \mathbf{y}$$

and therefore our solution is

$$\mathbf{x}^* = (A^T A)^{-1} A^T \mathbf{y}$$

which can also be written as

$$\mathbf{x}^* = A^+ \mathbf{y}$$

where A^+ is the **Pseudo-Inverse** of A .



Non-convex loss: Iterative optimisation

If the loss is not convex, we do not have algorithms that give us a direct solution, we need to use **iterative optimisation**

- **Iterative optimisation** involves making a series of steps in parameter space.
- There is a **current parameter vector** (or collection of them) which is adjusted at each iteration, hopefully decreasing the objective function, until optimisation terminates after some **termination criteria** have been met.

Iterative optimisation algorithm:

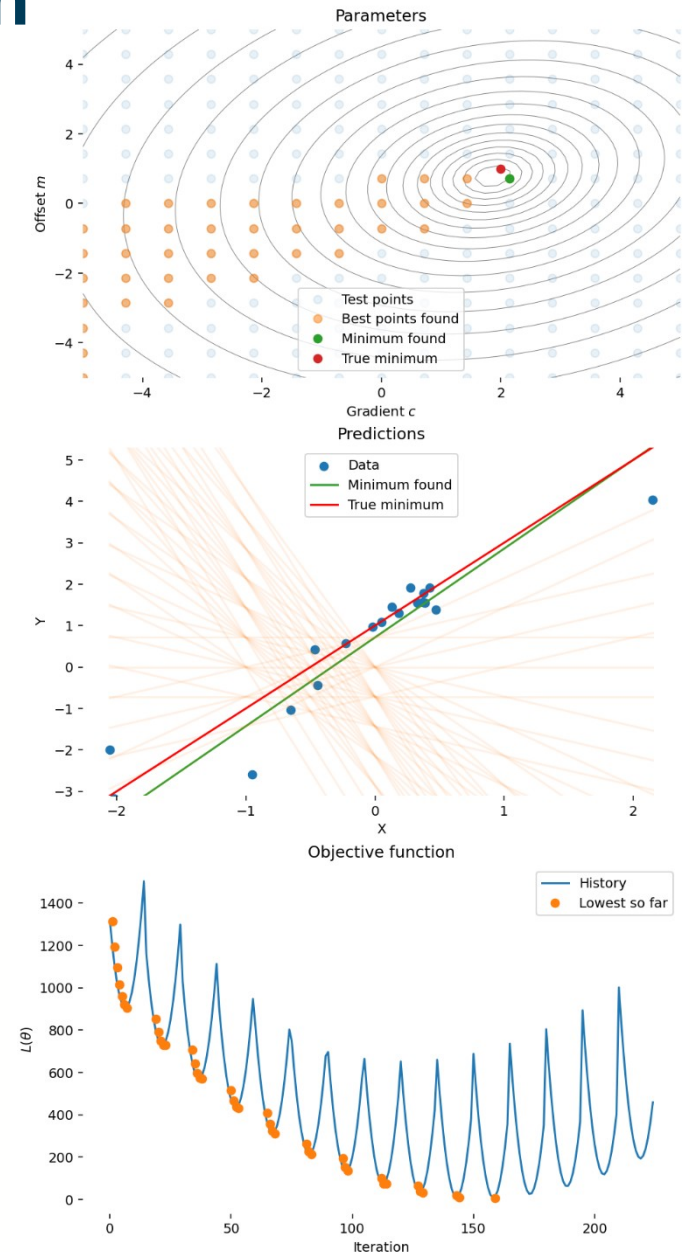
1. choose a starting point x_0
2. while objective function changing
 - adjust parameters
 - evaluate objective function
 - if better solution found than any so far, record it
3. return best parameter set found



Iterative optimisation - grid search

- **Regular search: grid search**
 - The parameter space is simply sampled by equally dividing the feasible set in each dimension, usually with a fixed number of divisions per dimension.
 - The objective function is evaluated at each θ on this grid, and the lowest loss θ found so far is tracked.

Grid search over possible line configurations





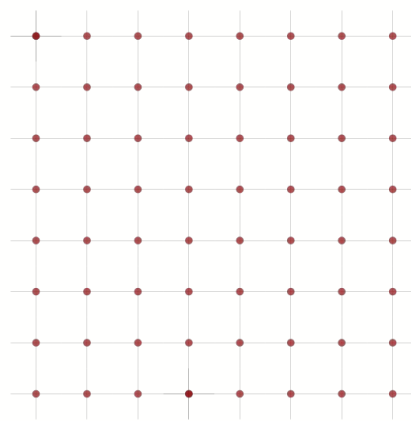
Iterative optimisation - grid search

- Revenge of the curse of dimensionality

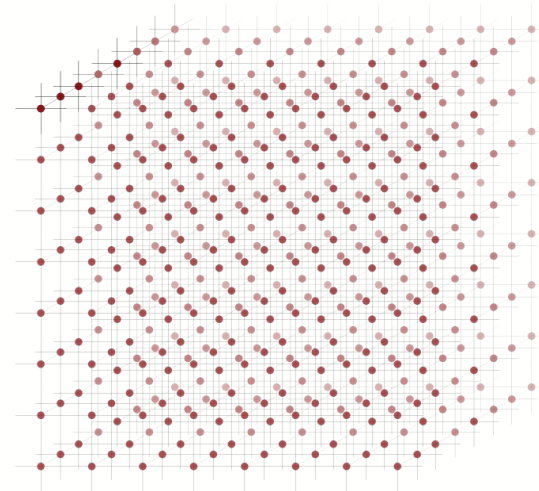
1 dimension



2 dimensions



3 dimensions



- While this is fine in 1D (just check 8 points) and 2D (just check 64 points), it breaks down completely if you have a 100 dimensional parameter space. This would need:

$8^{**}100 =$

2037035976334486086268445688409378161051468393665936250636140449354381299763336706183397376

evaluations of the objective function!



Iterative optimisation - grid search

Pros

- Works for any continuous parameter space.
- Requires no knowledge of the objective function.
- Trivial to implement.

Cons

- **Incredibly** inefficient
- Must specify search space bounds in advance.
- Highly biased to finding things near the "early corners" of the space.

We need better optimisation algorithms for larger problems!



Iterative optimisation - Simple stochastic: random search

- The simplest such algorithm, which makes *no* assumptions other than we can draw random samples from the parameter space, is **random search**.
- The process is simple:
 - Guess a random parameter θ
 - Check the objective function $L(\theta)$
 - If $L(\theta) < L(\theta^*)$ (the previous best parameter θ^*), set $\theta^* = \theta$
- There are many possibilities for a termination condition, such as stopping after a certain number of iterations after the last change in the best loss.



Iterative optimisation - Simple stochastic: random search

Pros

- Random search cannot get trapped in local minima, because it uses no local structure to guide the search.
- Requires no knowledge of the structure of the objective function - not even a topology.
- Very simple to implement.
- Better than grid search, almost always.

Cons

- *Extremely inefficient* and is usually only appropriate if there is no other mathematical structure to exploit.
- Must be possible to randomly sample from the parameter space (usually not a problem, though).
- Results do not necessarily get better over time. Best result might be found in the first step or a million steps later. There is no way to predict how the optimisation will proceed.



Meta-heuristics

There are a number of standard **meta-heuristics** than can be used to improve random search.

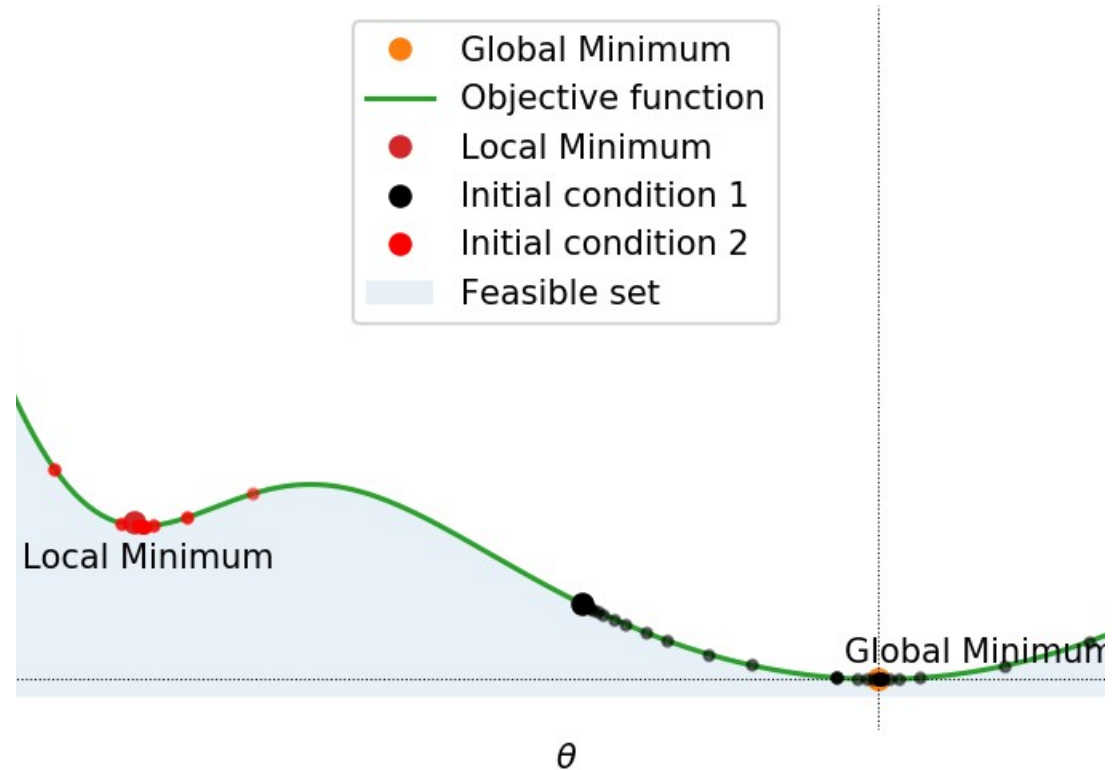
These are:

- **Locality** which takes advantage of the fact the objective function is likely to have similar values for similar parameter configurations. This assumes *continuity* of the objective function.
- **Temperature** which can change the rate of movement in the parameter space over the course of an optimisation. This assumes the existence of local optima.
- **Population** which can track multiple simultaneous parameter configurations and select/mix among them.
- **Memory** which can record good or bad steps in the past and avoid/revisit them.



Using locality - Hill climbing

- **Hill climbing: local search**
 - **Hill climbing** is a modification of random search which assumes some topology of the parameter space, so that there is a meaningful concept of a **neighbourhood** of a parameter vector; that we can make incremental changes to it.
 - **Simple hill climbing** adjusts just one of the parameter vector elements at a time, examining each "direction" in turn, and taking a step if it improves things.
 - **Stochastic hill climbing** makes a random adjustment to the parameter vector, then either accepts or rejects the step depending on whether the result is an improvement.





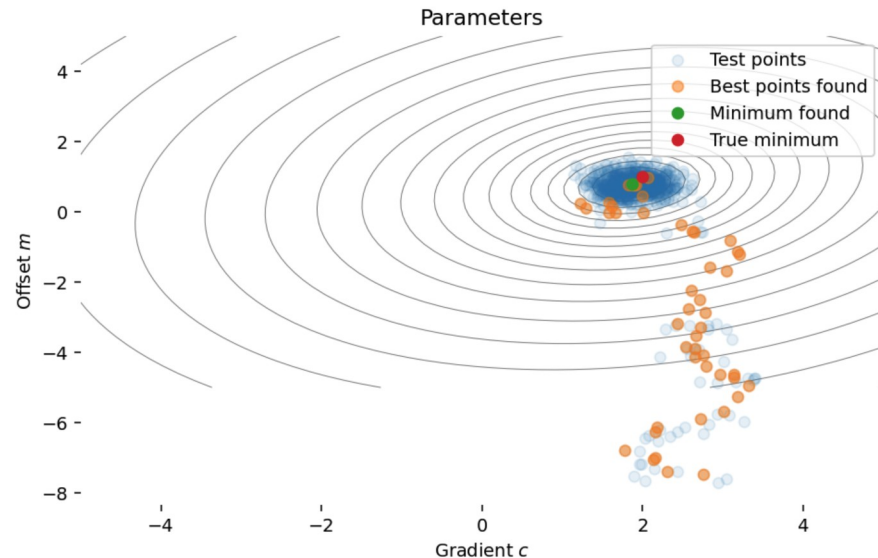
Using locality - Hill climbing

Pros

- Not much more complicated than random search
- Can be *much* faster than random search

Cons

- Hard to choose how much of an adjustment to make
- Can get stuck in minima
- Struggles with objective function regions that are relatively flat
- Requires that the objective function be (approximately) continuous



Locality + Temperature: Simulated annealing

- **Simulated annealing** extends hill-climbing with the ability to sometimes randomly go uphill, instead of always going downhill.
- It uses a **temperature schedule** that allows more uphill steps at the start of the optimisation and fewer ones later in the process. This is used to overcome ridges and avoid getting stuck in local minima.
- It allowing random "**bad jumps**" early process can help find a better overall configuration.

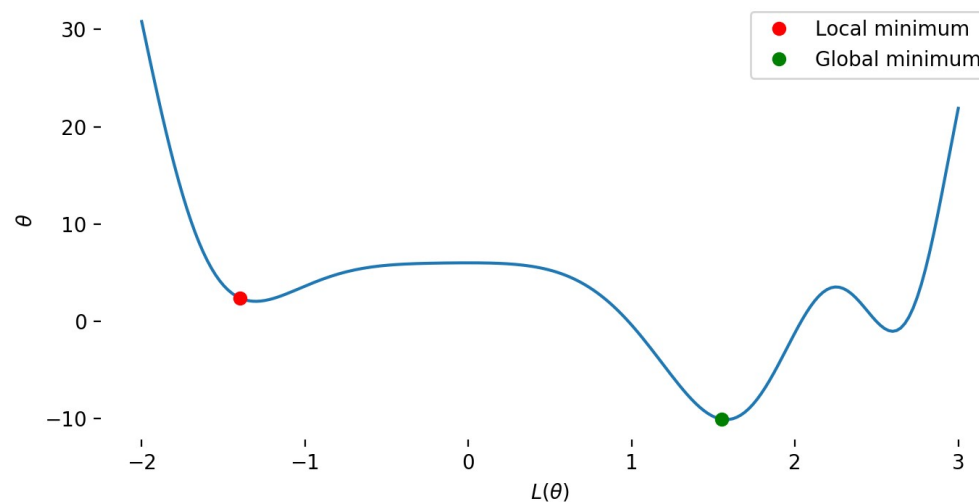


Image: hill climbing would get stuck in the local minimum at the left. Simulated annealing would sometimes accept "bad" local changes to ride over hills and get to a better minimum.



Locality + Temperature: Simulated annealing

- The "temperature schedule" comes from the idea of annealing metals.
- Molten metals have molecules bouncing around all over the place. As they **cool**, the random bouncing gets **smaller** and **smaller** as the molecules lock together into a tight lattice. Fast cooling results in less well structured metals than slow cooling.

Simulated annealing searching for a **maximum**. The objective here is to get to the highest point. In this example, it is **not enough** to use a simple hill climb algorithm, as there are **many local maxima**. By cooling the temperature slowly the global maximum is found.



Population - Genetic Algorithms

- Another nature-inspired variant of random search is to use a **population** of multiple competing potential solutions, and to apply some analogue of **evolution** to solving optimisation. This involves some of:
 - **mutation** (introducing random variation)
 - **natural selection** (solution selection)
 - **breeding** (interchange between solutions)
- This class of algorithms are often called **genetic algorithms** for obvious reasons.



Genetic Algorithms

- All genetic algorithms maintain some population of potential solutions (a set of vectors $\theta_1, \theta_2, \theta_3, \dots$), and some rule which is used to preserve some members of the population and cull others.
- The parameter set is referred to as the **genotype** of a solution.
- Simple population approaches
 - small random perturbations and a simple selection rule like "keep the top 25% of solutions, ordered by loss".
- Each iteration will perturb the solutions slightly by random mutation, cull the weakest solutions, then copy the remaining "fittest" solutions a number of times to produce the offspring for the next step. (random local search with population)



Genetic Algorithms - crossover

- **Crossover rules**

- crossover "merges" two possible parameter vectors θ_{mum} and θ_{dad} to form a new child parameter θ_{baby}
- Crossover works well when the parameter vector can be partitioned into distinct components, where offspring can plausibly inherit good qualities from both parents.
- It works less well when the crossover simply becomes a mishmash of parent qualities which average out.

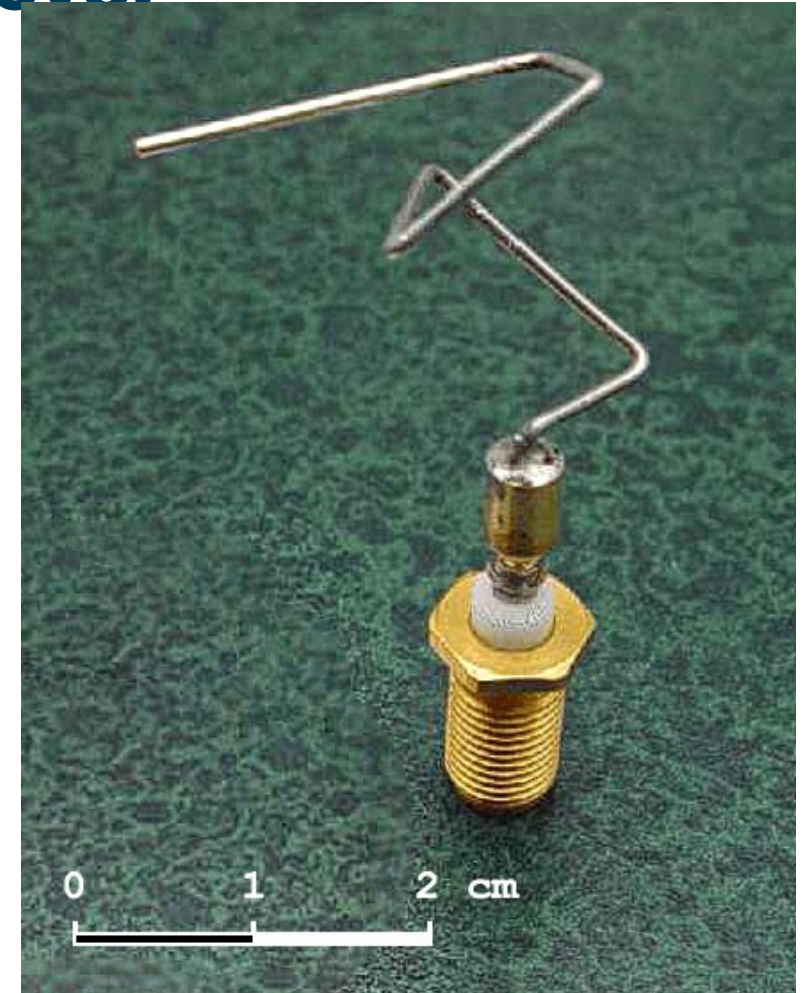


Image: The 2006 NASA ST5 spacecraft antenna. This complicated shape was found by an evolutionary computer design program to create the best radiation pattern. Source: [wikipedia](#), public domain



University
of Glasgow

Genetic algorithm

Animation from Flexible Muscle-Based Locomotion for Bipedal Creatures Geijtenbeek, T., van de Panne, M. & Stappen, A.F. van der (2013)



Genetic Algorithms

Pros

- Easy to understand and applicable to many problems.
- Requires only weak knowledge of the objective function
- Can be applied to problems with both discrete and continuous components.
- Some robustness against local minima, although hard to control.
- Great flexibility in parameterisation: mutation schemes, crossover schemes, fitness functions, selection functions, etc.

Cons

- Many, many "hyperparameters" to tune which radically affect the performance of the optimisation. How should you choose them?
- No guarantee of convergence; *ad hoc*.
- (Very) slow compared to using stronger knowledge of the objective function.
- Many evaluations of objective function are required: one per population member per iteration.



Memory+Population: Ant Colony Optimisation

Ant colony optimisation combines memory and population heuristics. It uses the idea of **stigmergy** to optimise problems:

stigmergy: A mechanism of spontaneous, indirect coordination between agents or actions, where the trace left in the environment by an action stimulates the performance of a subsequent action.
[wiktionary.org]



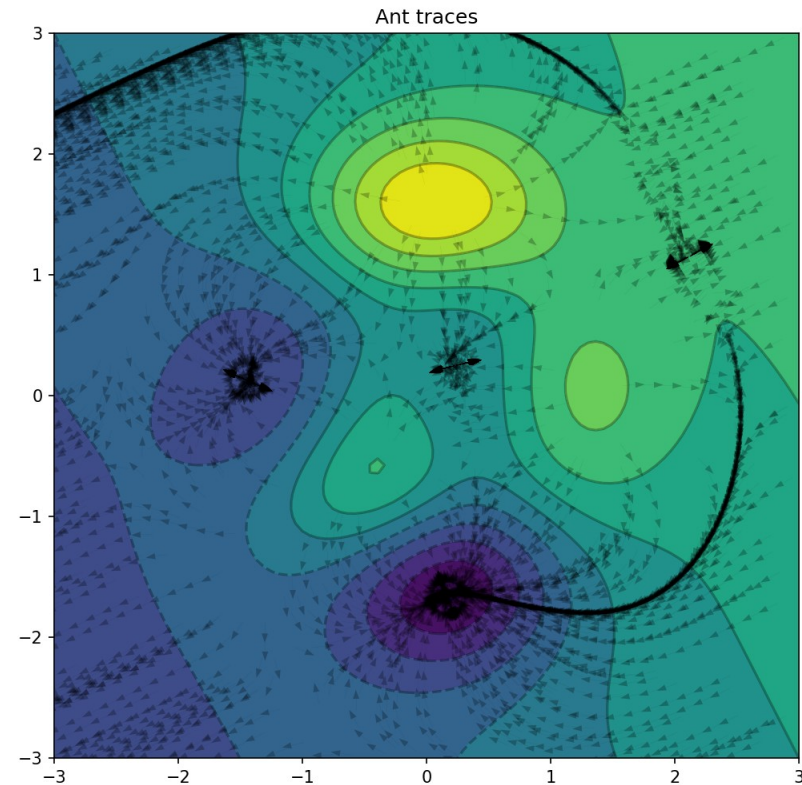


Ant Colony Optimisation

In terms of optimisation this means:

- having a population of parameter sets ("ants")
- having a memory of good paths through the space ("pheromones")

ACO is particularly well suited to path-finding and route-finding algorithms, where the memory structure of the pheromone trail corresponds to the solution structure.





Ant Colony Optimisation

Pros

- Can be very effective in spaces where good solutions are separated by large, narrow valleys.
- Can use fewer evaluations of the objective function than genetic algorithm if pheromones are effective.
- When it works, it really works.

Cons

- Moderately complex algorithm to implement.
- No guarantee of convergence; *ad hoc*.
- Even *more* hyperparameters than genetic algorithms.
- People think you work with ants.



Convergence

- An optimisation algorithm is said to **converge** to a **solution**.
- In convex optimisation, this means that the **global minimum** has been found and the problem is solved. In non-convex optimisation, this means a **local minimum** has been found from which the algorithm cannot escape.

Guarantees of convergence

- Some optimisation algorithms are guaranteed to converge if a solution exists; while others (like most heuristic optimisation algorithms) are not guaranteed to converge even if the problem has a solution.
- For example, a random search might wander the space of possibilities forever, never finding the specific configuration that minimises (or even reduces) the loss.



University
of Glasgow

Thank you for you attention!

**Contact:
Nicolas.Pugeault@Glasgow.ac.uk**